Bài viết báo cáo

1.Tổng quan:

-Thông tin cá nhân:Nguyễn Viết Bảo Nguyên
-MSV: 25022967
-Tổng quan thiết kế luồng thực thi:

main()
→ parseArgs() →đọc flag –gui,verbose..
→ Khởi tạo Renderer + Interaction
→ Khởi tạo Engine
→ engine.init()
→ engine.startGame()
→ chọn size / goal / mode / bot
→ engine.playGame()
→ [loop] showPlayer → getMove → displayBoard → check win
→ engine.endGame()
→ showResult
→ engine.close()

-Giao diện đồ họa sử dụng hình thức hiển thị:

- Font chữ: dùng SDL_ttf
  TTF_OpenFont("assets/font.ttf", FONT_SIZE);
  TTF_RenderText_Blended(font, text.c_str(), color);
- Vẽ hình học: dùng SDL primitive
  1. Bàn cờ:SDL_RenderDrawLine()
  2. Viền ô:SDL_RenderDrawRect()
  3. Symbol ‘X’: vẽ 2 đường chéo
     SDL_RenderDrawLine()

4. Symbol ‘O’ : vẽ hình tròn bằng các điểm
   SDL_RenderDrawPoint()

- Kiểu tương tác: Bàn phím(all input qua keyboard)
  1. Phím số:0-9(chọn goal,size,mode,bot level)
  2. ENTER để confirm
  3. BACKSPACE để delete

2.Chi tiết cài đặt:

1. Tư duy thiết kế

   +Luồng thực thi Engine:

   Engine::init() — khởi tạo renderer, interaction
   Engine::startGame() — chọn mode, size, goal, bot level, initboard
   Engine::playGame() — game loop chính
   Engine::endGame() — hiển thị kết quả
   Engine::close() — dọn tài nguyên

   +Sự thay đổi Engine so với lv1:

   -Level 1 là free functions,được gọi thẳng ko qua class nào

   -Level 2 Engine là class,gọi qua Interface,có vòng đời rõ ràng từ init() → startGame() → playGame() → endGame() → close()
2. Trả lời câu hỏi
3. Giao diện đồ họa: SDL2
   - Hiển thị:

     +Font chữ: TrueType dùng công thức để tính toán cỡ từng chữ:tiêu đề,thông báo,tọa độ ô

     +Vẽ hình học:

a. Lưới bàn cờ
b. Symbol “X”
c. Symbol “O”
d. Nền tối

+ Lý do chọn hình thức SDL2: nhẹ,tích hợp nhiều tính năng,có hỗ trợ Hardware-accelerated rendering(dùng GPU để vẽ thay vì CPU → vẽ nhanh hơn)

+ Hiệu năng:

```
[INFO] - [2026-05-16 16:48:03] [Engine] Playing game . . .
[DEBUG] - [2026-05-16 16:48:03] [Engine] starting turn #0
[DEBUG] - [2026-05-16 16:48:03] execution time of [bot#0->getMove()()] = 2.7e-06s
[DEBUG] - [2026-05-16 16:48:03] player 1 make move to (0, 2)
[DEBUG] - [2026-05-16 16:48:04] [Engine] turn done!
[DEBUG] - [2026-05-16 16:48:04] [Engine] starting turn #1
[DEBUG] - [2026-05-16 16:48:04] execution time of [bot#1->getMove()()] = 3.1e-06s
[DEBUG] - [2026-05-16 16:48:04] player 2 make move to (2, 2)
[DEBUG] - [2026-05-16 16:48:06] [Engine] turn done!
[DEBUG] - [2026-05-16 16:48:06] [Engine] starting turn #2
[DEBUG] - [2026-05-16 16:48:06] execution time of [bot#0->getMove()()] = 1.8e-06s
[DEBUG] - [2026-05-16 16:48:06] player 1 make move to (1, 2)
[DEBUG] - [2026-05-16 16:48:07] [Engine] turn done!
[DEBUG] - [2026-05-16 16:48:07] [Engine] starting turn #3
[DEBUG] - [2026-05-16 16:48:07] execution time of [bot#1->getMove()()] = 2.3e-06s
[DEBUG] - [2026-05-16 16:48:07] player 2 make move to (0, 0)
[DEBUG] - [2026-05-16 16:48:09] [Engine] turn done!
[DEBUG] - [2026-05-16 16:48:09] [Engine] starting turn #4
[DEBUG] - [2026-05-16 16:48:09] execution time of [bot#0->getMove()()] = 1.8e-06s
[DEBUG] - [2026-05-16 16:48:09] player 1 make move to (1, 1)
[DEBUG] - [2026-05-16 16:48:10] [Engine] turn done!
[DEBUG] - [2026-05-16 16:48:10] [Engine] starting turn #5
[DEBUG] - [2026-05-16 16:48:10] execution time of [bot#1->getMove()()] = 2.1e-06s
[DEBUG] - [2026-05-16 16:48:10] player 2 make move to (2, 1)
[DEBUG] - [2026-05-16 16:48:12] [Engine] turn done!
[DEBUG] - [2026-05-16 16:48:12] [Engine] starting turn #6
[DEBUG] - [2026-05-16 16:48:12] execution time of [bot#0->getMove()()] = 2.6e-06s
```

- Tương tác: Trò chơi xử lý người dùng bằng cách nhận dữ liệu từ bàn phím
  + Phím số:0-9
  + Phím Enter
  + Phím Backspace
  + 1/2/3 để chọn mode/bot level

Khó khăn lớn nhất trong quá trình cài đặt tương tác đó chính là sử dùng thư viện SDL bởi mỗi lần bấm

thì phải bắt từng phím,lưu buffer,redraw để vẽ lại toàn bộ màn hình

Câu hỏi yêu cầu:

Câu hỏi 1:Trò chơi thể hiện đầy đủ 4 nguyên lý OOP

1.Đóng gói:Private members ko truy cập trực tiếp,chỉ qua getter/setter

```
SDL_Renderer* renderer = nullptr;
TTF_Font* s_font;
int g_player = 0;
SDL_Renderer* getSDLRenderer() { return renderer; }
void setPlayer(int player, bool is_bot);
```

2.Kế thừa:

I_Renderer ← SDLRenderer, TerminalRenderer
I_Interaction ← SDLInteraction, TerminalInteraction
Bot ← BotLv1, BotLv2, BotLv3

3.Đa hình:

Engine dùng cùng interface, hành vi khác nhau tùy implementation

```
I_Renderer* r = new SDLRenderer();
r->displayBoard(...);

I_Renderer* r = new TerminalRenderer();
r->displayBoard(...);
```

4. Trừu tượng:

```
class I_Renderer {
   virtual void displayBoard(...) = 0;
   virtual void showResult(...)  = 0;
   virtual void showPlayer(...)  = 0;
};
```

Câu hỏi 2:Thiết kế của em vẫn còn chút khó khăn để cân bằng 2 giao diện Terminal và giao diện GUI bởi SDLInteraction cần vẽ UI khi nhập liệu mà Terminal thì ko cần → SDLInteraction phụ thuộc SDLRenderer* thay vì I_Renderer*

Câu hỏi 3:Mức độ tuân thủ SOLID:Thiết kế của e hầu hết có tuân thủ nguyên tắc SOLID nhưng trong số đó vẫn còn 1 phần nhỏ vi phạm

1. **Single Responsibility**
   Vi phạm 1 phần:SDLRenderer vừa vẽ lên màn hình vừa lưu game state
   → Render chỉ nên vẽ còn lưu game thì nên thuộc Engine hoặc 1 class nào khác

Trong renderer.h - private members của SDLRenderer

```
char s_board_data[BOARD_N_MAX][BOARD_N_MAX];
int s_board_size = 0;
int g_player = 0;
bool g_is_bot = false;
```

- Vừa lưu Board
- Vừa lưu bot
- Vừa lưu trạng thái player

2. **Open/Closed**
   Tuân thủ
   -Thêm bot mới chỉ cần tạo class mới kế thừa Bot, không sửa code cũ

```
class BotLv1 : public Bot { ... };
class BotLv2 : public Bot { ... };
class BotLv3 : public Bot { ... };
```

3. **Liskov Substitution**

Tuân thủ

if (!config.gui_flag)

iRenderer = new TerminalRenderer();

iRenderer = new SDLRenderer();

→Thay iRenderer bằng SDLRenderer* hoặc TerminalRenderer*

- Engine nhận iRenderer và chạy

engine = new Engine(&config, iRenderer, iInteraction);

engine->playGame(); → chạy đúng dù Terminal hay SDL

4. **Interface Segregation**

Tuân thủ

-Tách riêng 2 interface, không gộp chung

class I_Renderer { ... } chỉ render

class I_Interaction { ... } chỉ input

5. **Dependency Inversion**

Vi phạm 1 phần: SDLInteraction phụ thuộc vào SDLRenderer thay vì I_Renderer*(interface)

→ phải là I_Renderer* renderer

-Trong interaction.h

class SDLInteraction : public I_Interaction {

private:

SDLRenderer* renderer; // ❌ không phải I_Renderer*

};

Nguyên nhân:SDLInteraction cần gọi redrawBoard(),getFont(),getPlayer() mà các hàm này lại ko có trong I_Renderer nên mới phải vi phạm như vậy

Câu hỏi 4:

Chương trình của e có sử dụng thêm Delegation Pattern:

static void renderMove(SDLRenderer* r, ...) {

…

r->redrawBoard(); →Cho phép SDLRenderer vẽ

…

itxt(rend, font, playerText, 400, 630, white);

itxt(rend, font, "Enter move:", 400, 680, white);

itxt(rend, font, "Move: [" + buf_row + "]", 400, 730, white);

}

```
static void renderMove(SDLRenderer* r, const std::string& buf_row, const st
    SDL_Renderer* rend = r->getSDLRenderer();
    TTF_Font* font = r->getFont();
    TTF_Font* font_small = r->getFontSmall();
    if (!rend) return;
    r->redrawBoard();

    SDL_Rect bg = {0, 610, 800, 220};
    SDL_SetRenderDrawColor(rend, 20, 20, 20, 255);
    SDL_RenderFillRect(rend, &bg);

    SDL_Color white = {255, 255, 255, 255};
    SDL_Color gray  = {160, 160, 160, 255};
    itxt(rend, font, "TIC TAC TOE", 400, 70, white);
    std::string playerText;
    if(r->getIsBot()) playerText = std::format("Bot {} is thinking...", r->
    else playerText = std::format("Player {} turn", r->getPlayer()+1);
    itxt(rend, font, playerText, 400, 630, white);
    itxt(rend, font, "Enter move:", 400, 680, white);
    itxt(rend, font, "Move: [" + buf_row + (buf_col.empty() ? "" : ", " + b
    itxt(rend, font_small, "<ENTER> DE DONG Y", 400, 780, white);
    SDL_RenderPresent(rend);
}
```

Ý tưởng:SDLInteraction sẽ ko tự vẽ lại board,thay vào đó gọi r→redrawBoard() để SDLRenderer vẽ

3.Kết quả thực nghiệm:

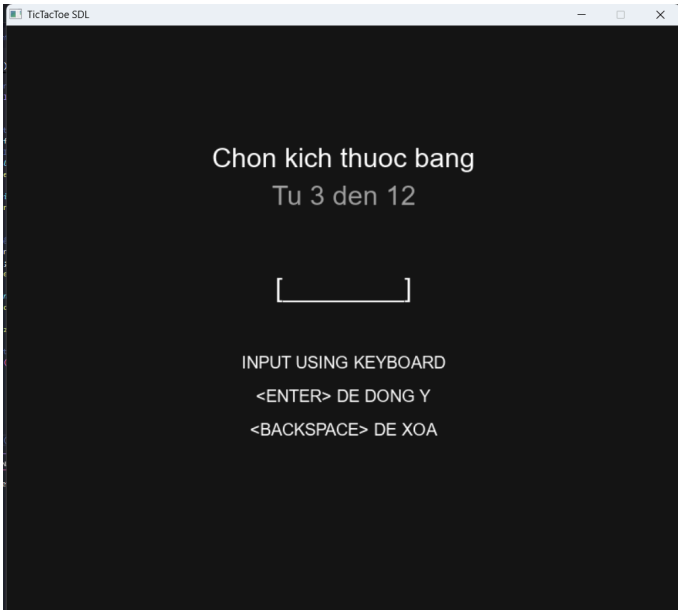
Ảnh chụp màn hình

Tiêu chí	Terminal	SDL
Hiển thị Board	Dùng bằng ký tự | và - để vẽ	Vẽ đường thẳng thực,X/O bằng đồ họa vector
Nhập liệu	Gõ số hàng/cột rồi Enter/Backspace	Gõ tọa độ hàng cột rồi Enter/Backspace
Tốc độ	Nhanh,nhẹ	Cần khởi tạo SDL,nặng hơn
Trải nghiệm	Quen thuộc với lập trình viên	Quen thuộc với người dùng phổ thông

Menu

```
>----- Tic-tac-toe [Console v0.4.25022967] -----<

Size Input(NxN,3 <= N <= BOARD_N_MAX): 5
Goal Input(3 - 5, goal <= size): 3
(1) PvP | (2) PvE | (3) EvE: 2
(1) EASY | (2) MEDIUM | (3) HARD: 1
```



Board

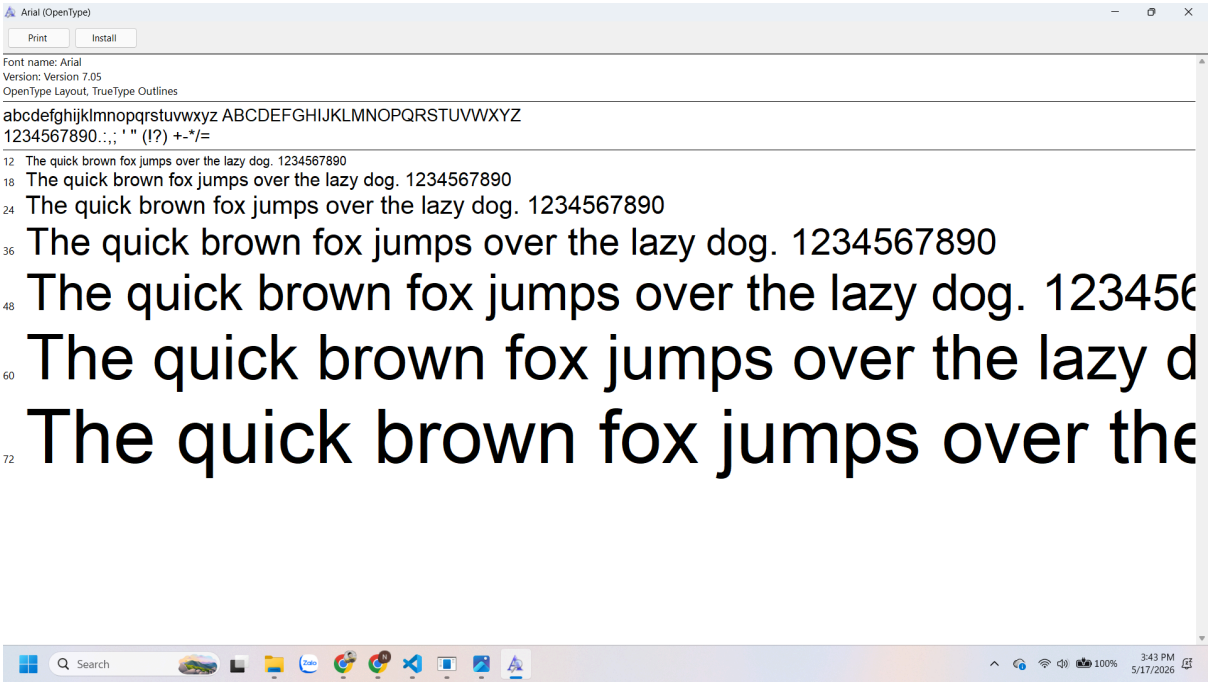
```
   0 1 2 3 4
  -----------
0 |- - - - -
1 |- - - - -
2 |- - - - -
3 |- - - - -
4 |- - - - -
```



→ Giao diện Terminal đơn giản,dễ debug.Giao diện SDL có cái nhìn trực quan hơn,dễ nhìn,tăng trải nghiệm cho người chơi vì thế đòi hỏi code nhiều hơn và xử lý nhiều thứ hơn

Tài nguyên game(Assets)

1: Raw asset — File font.ttf xem trong Windows Font Viewer



Quy trình render:
TTF_OpenFont() → TTF_RenderText_Blended() → SDL_CreateTextureFromSurface() → SDL_RenderCopy()

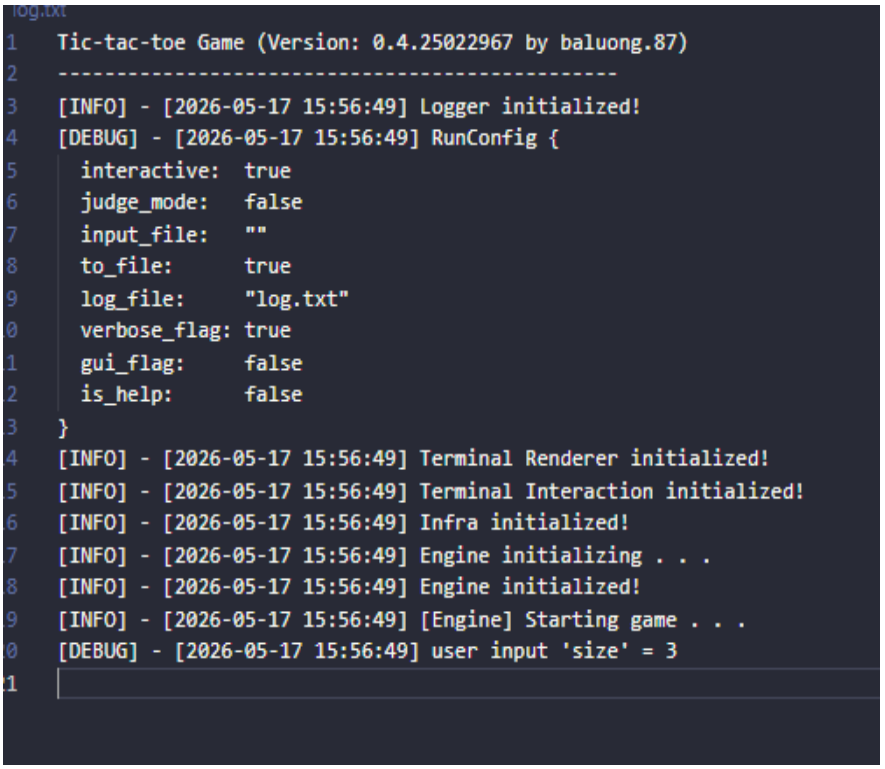
2: Kết quả sau render — Font hiển thị trong game SDL

Trích dẫn Log:

log.txt

```
Tic-tac-toe Game (Version: 0.4.25022967 by baluong.87)
-------------------------------------------------
[INFO] - [2026-05-17 15:50:25] Logger initialized!
[INFO] - [2026-05-17 15:50:25] SDL Renderer initialized!
[INFO] - [2026-05-17 15:50:25] SDL Interaction initialized!
[INFO] - [2026-05-17 15:50:25] Infra initialized!
[INFO] - [2026-05-17 15:50:25] Engine initializing . . .
[INFO] - [2026-05-17 15:50:25] Engine initialized!
[INFO] - [2026-05-17 15:50:25] [Engine] Starting game . . .
[INFO] - [2026-05-17 15:50:32] Board initialized!
[INFO] - [2026-05-17 15:50:32] [Engine] Game started!
[INFO] - [2026-05-17 15:50:32] [Engine] Playing game . . .
[INFO] - [2026-05-17 15:50:42] [Engine] Game done!
[INFO] - [2026-05-17 15:50:42] [Engine] Ending game . . .
[INFO] - [2026-05-17 15:50:48] [Engine] Game ended!
[INFO] - [2026-05-17 15:50:48] Engine closing . . .
[INFO] - [2026-05-17 15:50:48] Engine closed!
[INFO] - [2026-05-17 15:50:48] Engine destroyed!
[INFO] - [2026-05-17 15:50:48] Infra destroyed!
[INFO] - [2026-05-17 15:50:48] Logger closing . . .
```

Với SDL log



Với Terminal log

4.Nhận xét và tổng kết bài làm

- **Về phong cách lập trình:**
  - Điểm mạnh:
    + Tái sử dụng code mà ko cần viết lại
    + Dễ mở rộng
    + Dễ bảo trì
  - Điểm yếu:
    + Khó debug
    + Phụ thuộc vào nhau

-Khi game phức tạp hơn–OOP có dễ bảo trì hơn vì mỗi file,class đều làm tách biệt nhau,khi cần thay đổi hay sửa code thì cũng ko cần sửa điều chỉnh toàn bộ code

-Ko phải dự án nào cũng phù hợp với phong cách lập trình này,chỉ có những dự án lớn,nhiều người làm và cần mở rộng lâu dài mới phù hợp

- **Về giao diện người dùng:**

- Điểm thú vị nhất theo em thấy là phần vẽ bảng,lên màu,X/O

- Điểm khó khăn nhất đó chính là vẽ hình tròn(vì ko có hàm vẽ hình tròn riêng nên phải vẽ đường cong,vẽ từng điểm),vẽ bảng(phải clear bảng mỗi lần bấm phím để hiện board mới)

→So với SDL thì e thích Terminal hơn tại vì Terminal giao diện đơn giản,dễ debug,phù hợp lập trình viên,ko phụ thuộc vào các thư viện SDL,tập trung vào logic hơn thay vì renderer,chạy vẫn mướt mườn mượt

- **Về tổ chức mã nguồn:**

Cách tổ chức file:

game/ — logic, engine, bot
sdl/ — giao diện đồ họa
terminal/ — giao diện terminal
utils/ — config, logger, helper

Mỗi file đều có header comment riêng:điều này giúp cho mỗi người truy cập file có thể hiểu nhanh chóng các hàm nói gì và cần làm gì,biết ngay file nào chứa gì,nhiều người có thể làm song song với nhau khi đc chia thành nhiều file như vậy

-1 file:Dự án nhỏ,bài tập đơn giản

-Nhiều file:dự án có nhiều file độc lập,nhiều người cùng làm,bảo trì lâu dài

- **Về thiết kế chương trình**

  Vai trò:

  +AI:Viết code nhanh,gợi ý,check và giải thích đc lỗi

  +Human:Xác định bài toán,quyết định cấu trúc,thiết kế

-Con người chưa thể bị thay thế được:vì AI còn hiểu yêu cầu mơ hồ,ko giống cách chúng ta suy nghĩ,phải đưa ra lời dẫn rất nhiều mới có thể đúng ý đc,chưa sáng tạo được nếu gặp bài toán mới

-Vấn đề của các công cụ AI hiện tại:

1. Dễ bị phụ thuộc → ko hiểu code nếu ko xem lại
2. Ko hiểu ngữ cảnh →có thể viết đúng code nhưng sai câu trúc tổng thể
3. Tự đưa ra giải pháp sai,tự sửa

- **Bài học rút ra:**

  Khó khăn lớn nhất mà e đã gặp đó chính là lần đầu tiên sử dụng SDL,để mà nói thì em vẫn chưa thành thạo SDL lắm nên chỉ vượt qua 1 phần,e chỉ là newbie nên cái gì cũng thấy khó khăn😔về lập trình thì lv2 này cũng là ánh xạ từ lv1 nên toàn bộ khúc mắc nều nằm ở SDL

5.Ghi chú sử dụng AI

AI bản thân sử dụng:Claude

- Mục đích sử dụng:

  Như em có nói ở trên thì lv2 này khó khăn nằm ở SDL và gần như e chưa biết gì,nên việc sử dụng AI ở phần này cũng giảm ít khó khăn trong quá trình hoàn

thiện project này,việc vẽ bảng,X/O thì AI đều support cho em bằng cách e bảo Claude viết pesudo code 1 cách ngắn gọn,từ đó e sẽ flow theo để viết,cứ mỗi lần viết xong e sẽ nhờ nó check lỗi 1 lượt,hễ sai ở đâu thì e sửa ở đó

- Mức độ đóng góp của AI vào project này: 45-50%

6.Nguồn tham khảo

1.Youtube(kênh YT:The Builder)
2.SDL2 Documentation
3.Claude AI